UNIVERSITÄT TÜBINGEN
PROF. DR.-ING. HENDRIK P.A. LENSCH
LEHRSTUHL FÜR COMPUTERGRAFIK
PHILIPP LANGSTEINER (PHILIPP.LANGSTEINER@UNI-TUEBINGEN.DE)
MEHRAN HOSSEINZADEH (MEHRAN.HOSSEINZADEH@UNI-TUEBINGEN.DE)
DEBORAH KORNWOLF (DEBORAH.KORNWOLF@STUDENT.UNI-TUEBINGEN.DE)

MARCH 8, 2026

MASSIVELY PARALLEL COMPUTING
ASSIGNMENT 5

Instructions

Present your results to the exercise instructors to get a grading on this exercise sheet. You can earn one **BEE** by presenting all exercises on this sheet.

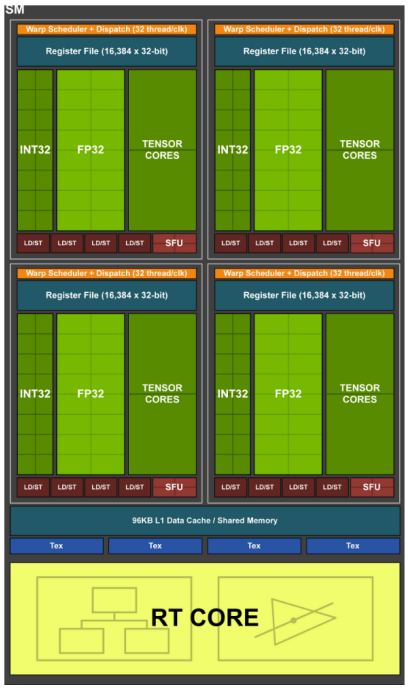
5.1 Matrix Multiplication (40 P)

- Start with the source code in the `MatrixMul` folder.
- Implement two square matrix multiplications:
  - `multiplyMatrixGpuBasic(...)` should calculate one element per thread by reading the input data trivially from global memory.
  - `multiplyMatrixGpuSharedMemory(...)` should also calculate one element per thread, but blocks should cooperate to read common input data to shared memory first.

5.2 Matrix Multiplication Using Tensor Cores (60 P)

Before you start with this exercise, make sure that for this exercise you connect to a machine with a **2080Ti**, as the 1080Ti and TITAN do not have Tensor Cores. You could also keep programming on one of the other machines and just run your code on a 2080Ti.
Start with the source code in the `TensorCores` folder. Your task is to implement a square matrix multiplication using Tensor Cores by using warp matrix functions from NVIDIA.

As you can see in Figure 1, several Tensor Cores are located on each SM, 4 per SM on the newer Hopper architecture and 8 (but smaller ones) per SM on the Turing architecture we are working on. Newer architectures offer more features like a Tensor Memory Accelerator (TMA) to even further decrease the memory access time, however, we need to work with what we have available on our GPUs. Having those Tensor Cores on the SM allows us to directly perform the matrix multiply operation on a Tensor rather than single elements and storing and accumulating intermediate results, which saves a lot of register accesses.
In practise, later in your career you will most likely be using Tensor Cores through some library like cuBLAS or cuDNN. However, a lot of the details we want to understand in this course are abstracted there, which is why we will use NVIDIA's Tensor Core programming API which is defined in the `mma.h` header in the `nvcuda::wmma` namespace.



(a) Streaming Multiprocessor of the Turing architecture (RTX 2080Ti)

(b) Streaming Multiprocessor of the Hopper architecture (H100)

Figure 1

Tensor Cores accelerate general matrix multiplication (GEMM) problems of the form:

$$D = A \cdot B + C \tag{1}$$

where $A \in \mathbb{R}^{m \times k}, B \in \mathbb{R}^{k \times n}, C \in \mathbb{R}^{m \times n}$ and $D \in \mathbb{R}^{m \times n}$. However, the Tensor Cores are comparably small and generally speaking an entire matrix won't fit into the Tensor Core. Therefore, we need to perform a matrix multiplication decomposition and partition the matrices into fragments (NVIDIA speak). The idea behind this partitioning is already familiar to you from the square matrix multiplication using shared memory.

Now, you should implement the multiplication of two square matrices ($m = n = k$). We will set $C = 0$ to observe the result of a plain square matrix multiplication even though the Tensor Core performs the GEMM.

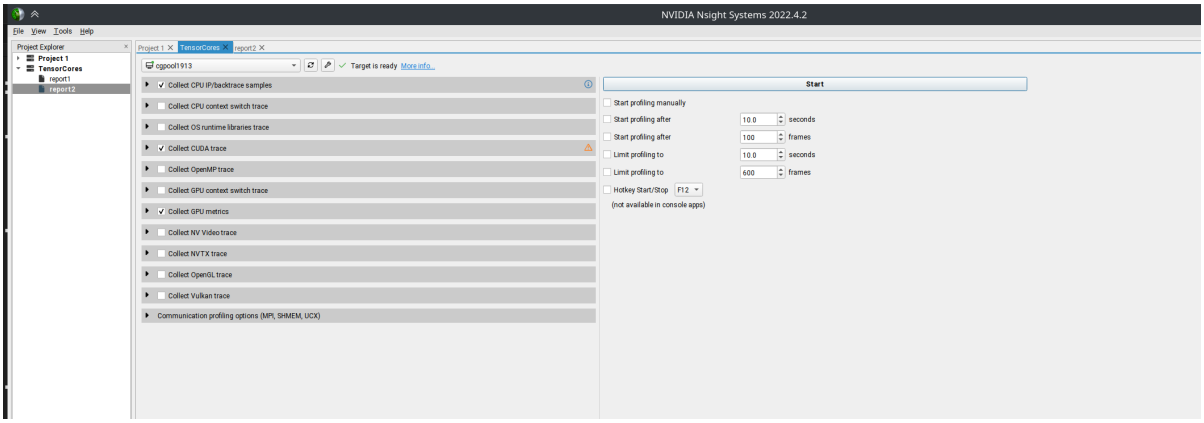
- start from the `main.cu` file in the `TensorCores` folder.
- familiarize yourself with the kernel launch parameters of the `TensorCoreMatrixMultiplication(...)`. Note, that the `wmma` works on a warp level, so 32 threads.
- note, that the multiplicands are of half precision, while the output is full precision. The CUDA Programming Guide tells us which datatype combinations and fragment configurations are possible.
- in the `TensorCoreMatrixMultiplication(...)` kernel, declare the fragments for all four matrices A, B, C and D ($m = n = k = S$, `S` is defined as macro!), make sure to set the proper `Use` parameter in the template and determine the correct layout to be used (`nvcuda::wmma::col_major` or `nvcuda::wmma::row_major`)
- load the multiplicand data into the fragments for the multiplicands

- perform the matrix multiplication
- note, that the example from the lecture only showed a 16×16 matrix multiplication and you now need to "move" the block over the k dimension
- then, load the output matrix fragment and store the accumulated result into the output matrix fragment
- note, the Tensor Core computes a multiplication on a 16×16 fragment and we only want to store the first 16×16 slice of the fragment in our output matrix.
- store the output fragment into the output matrix.

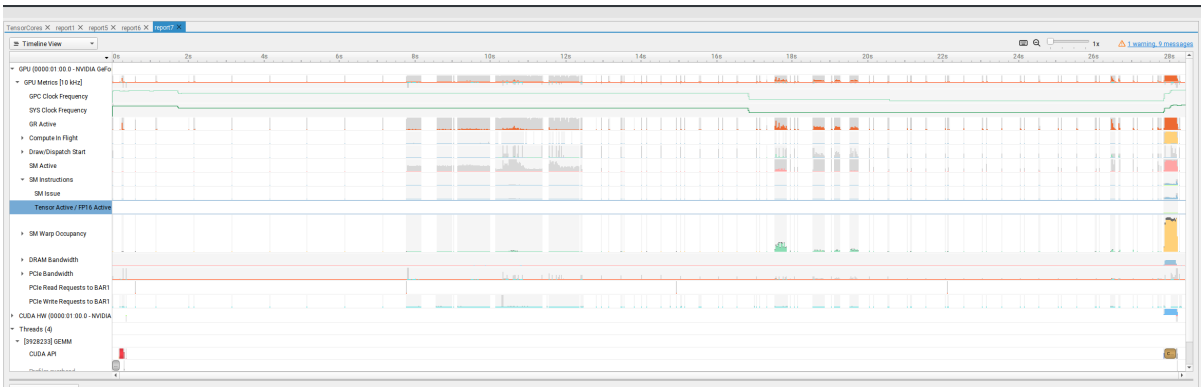
Now you can see the speedup compared to the CPU implementation, however, we also want you to check whether the Tensor Cores really were active:

- Open Nsight Systems and select a target machine for profiling (the one you are connected to)
- Check the Boxes for Collect CPU IP/backtrace samples, Collect CUDA trace and Collect GPU Metrics; uncheck the others (See Subfigure 2a)
- Wait until the report is created, it should look life Subfigure 2b
- Zoom into the area where the `TensorCoreMatrixMultiplication(...)` kernel was running (See Subfigure 2c, don't be surprised by the short kernel time, most of the program time is consumed by the CPU computation).
- The result should look approximately like Subfigure 2d. (Don't be surprised by the low utilization, we would activate the cores more if we would utilize the column-major trick, prepare the data in a nice layout already... for now you should primarily see, that the cores are active, but in the future there is a lot you can and should do to achieve a higher utilization.)

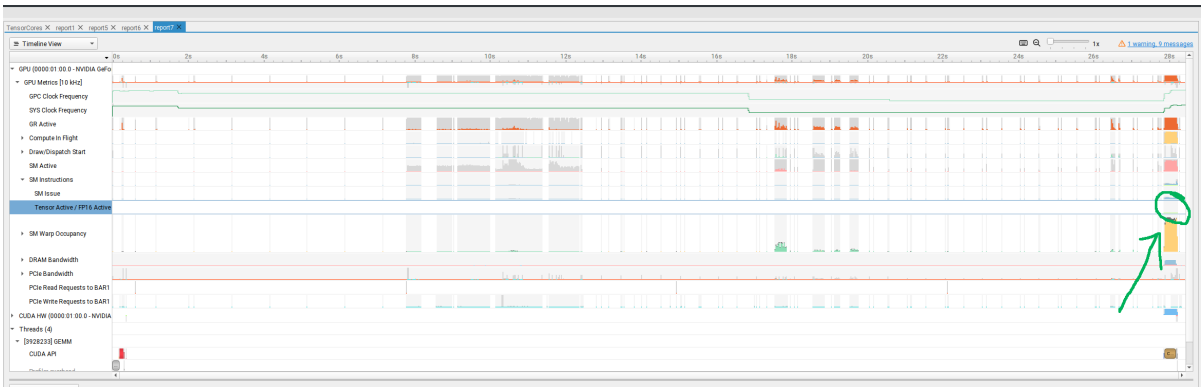
You did it :).


NVIDIA Nsight Systems 2022.4.2
File View Tools Help
Project Explorer
Project 1
TensorCores
report1
report2
Target is ready
Collect CPU IP/backtrace samples
Collect CPU context switch trace
Collect OS runtime libraries trace
Collect CUDA trace
Collect OpenMP trace
Collect GPU context switch trace
Collect GPU metrics
Collect NV Video trace
Collect NVTX trace
Collect OpenGL trace
Collect Vulkan trace
Communication profiling options (MPI, SHMEM, UCX)
Start
Start profiling manually
Start profiling after
Start profiling after
Limit profiling to
Limit profiling to
Hotkey Start/Stop
(not available in console apps)

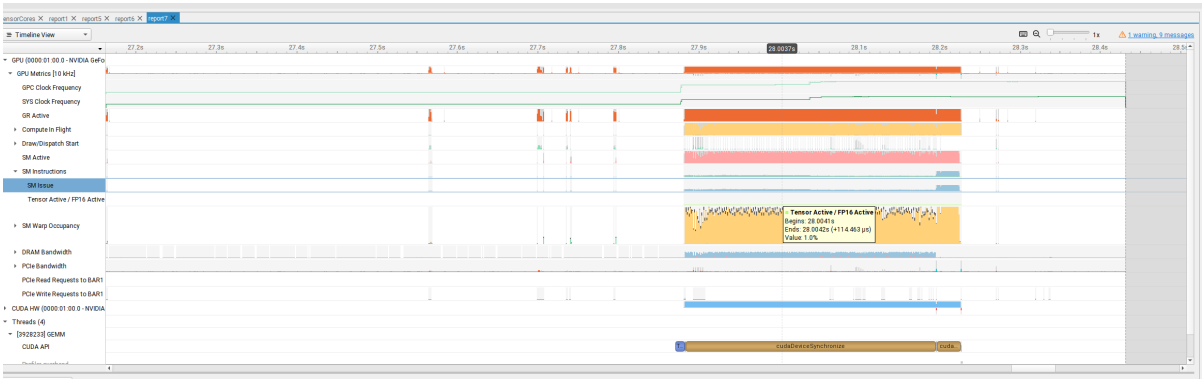
(a)


Timeline View
GPU Metrics [10 kHz]
GPC Clock Frequency
SYS Clock Frequency
GR Active
Compute In Flight
Draw/Dispatch Start
SM Active
SM Instructions
SM Issue
Tensor Active / FP16 Active
SM Warp Occupancy
DRAM Bandwidth
PCIe Bandwidth
Threads (4)
CUDA API

(b)


Timeline View
GPU Metrics [10 kHz]
GPC Clock Frequency
SYS Clock Frequency
GR Active
Compute In Flight
Draw/Dispatch Start
SM Active
SM Instructions
SM Issue
Tensor Active / FP16 Active
SM Warp Occupancy
DRAM Bandwidth
PCIe Bandwidth
Threads (4)
CUDA API

(c)


Timeline View
GPU Metrics [10 kHz]
GPC Clock Frequency
SYS Clock Frequency
GR Active
Compute In Flight
Draw/Dispatch Start
SM Active
SM Instructions
SM Issue
Tensor Active / FP16 Active
SM Warp Occupancy
DRAM Bandwidth
PCIe Bandwidth
Threads (4)
CUDA API
Tensor Active / FP16 Active

(d)